

CI/CD (Continuous Integration and Continuous Deployment/Delivery) is a set of practices and principles in **DevOps** that enable software development teams to deliver code changes more frequently, reliably, and with fewer errors.

What is CI/CD?

<u>Term</u>	<u>Description</u>
CI - Continuous Integration	Developers merge code changes frequently into a shared repository, where automated builds and tests are run.
CD - Continuous Delivery	Ensures that code is automatically prepared for a release to production after passing CI. Manual trigger is used to deploy.
CD - Continuous Deployment	Extends continuous delivery by automatically deploying code to production once it passes all stages.

CI/CD Pipeline: Deep Dive into the Process

Here's a step-by-step overview of a typical CI/CD pipeline:

1. Source Code Management (SCM)

- Developers push code to a version control system like **Git** (GitHub, GitLab, Bitbucket).
- A git push or merge triggers the pipeline.

2. Build Stage (CI)

- Code is compiled or built into executable packages.
- Build tools: **Maven, Gradle, npm, Webpack, Docker**, etc.
- Also includes:
 - Dependency installation
 - Code linting and static analysis
 - Unit testing

 Output: Build artifacts (JAR, WAR, Docker images)

3. Test Stage

Logging: ELK Stack, Splunk, Fluentd

Alerting: PagerDuty, Opsgenie

AWS Service	Role in CI/CD	Description
CodeCommit	Source Control	Git-based source code repository
CodeBuild	Build	Build and test your code in the cloud
CodePipeline	Orchestration	Automates the CI/CD workflow
CodeDeploy	Deployment	Deploys code to EC2, Lambda, or on-prem servers

Benefits of CI/CD

- Faster and more reliable releases
- Immediate feedback and rapid bug detection
- Reduced integration risk
- Better developer productivity
- Consistent deployment process
- Easier rollbacks and monitoring

 Real-World CI/CD Example

Imagine you're building a Node.js web app and using GitHub + Docker + AWS:

1. Developer pushes code to **GitHub**
2. **GitHub Actions** triggers:
 - Build Docker image

- Automated tests run to validate the code:
 - **Unit Tests** – test small code units.
 - **Integration Tests** – test interactions between components.
 - **End-to-End (E2E) Tests** – simulate user workflows.
 - **Security Tests** – check for vulnerabilities (e.g., OWASP, Snyk).
 - **Performance Tests** – test load and scalability.

✓ Goal: Detect issues early in the lifecycle.

4. Artifact Storage

- Artifacts are stored for deployment in:
 - **Artifact repositories:** Nexus, JFrog Artifactory
 - **Container registries:** Docker Hub, Amazon ECR, Google Container Registry (GCR)

5. Staging Deployment (Continuous Delivery)

- Artifacts are deployed to a **staging environment**.
- Smoke tests and UAT (User Acceptance Testing) can occur here.
- Manual or automated approvals can be configured.

6. Production Deployment (Continuous Deployment)

- If all tests pass and approvals are in place, the code is **automatically deployed** to production.
- Strategies:
 - **Blue-Green Deployment**
 - **Canary Releases**
 - **Rolling Updates**
 - **Feature Toggles**

7. Monitoring & Feedback

- After deployment, **real-time monitoring** tools track application health.
- Tools:
 - **Application Monitoring:** New Relic, Dynatrace, Datadog

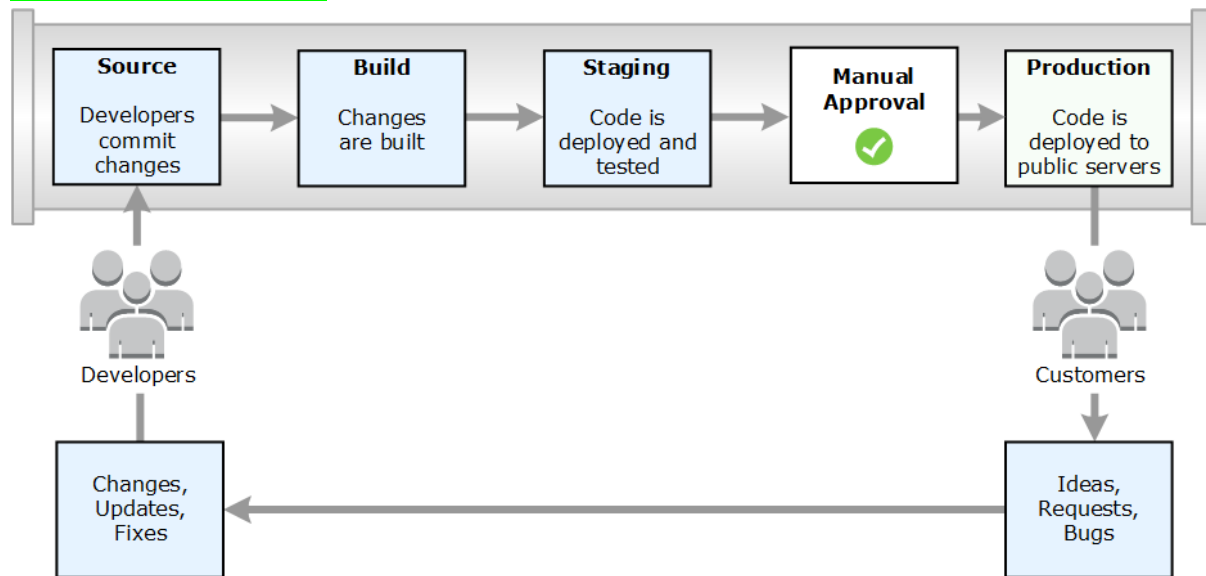
- Run unit and integration tests
 - Push Docker image to **Amazon ECR**
 - **AWS CodePipeline** pulls the image and deploys to **ECS (Elastic Container Service)**
 - Run unit and integration tests
 - Push Docker image to **Amazon ECR**
 - **AWS CodePipeline** pulls the image and deploys to **ECS (Elastic Container Service)**
- Monitoring with **CloudWatch** and alerts via **SNS**

Putting It All Together – AWS CI/CD Flow

Imagine you're deploying a web app:

1. **CodeCommit** stores your source code.
2. A push to the repo triggers **CodePipeline**.
3. **CodePipeline** uses **CodeBuild** to:
 - Install dependencies
 - Run tests
 - Build a production-ready version
4. **CodePipeline** sends the output to **CodeDeploy**.
5. **CodeDeploy** deploys the code to a fleet of EC2 instances or Lambda functions.

CI/CD PROCESS



Step 1: Setup GitHub Repository (Source Stage)

1. Go to GitHub → Create new repository
 - Name it (e.g. my-python-app-ci)
 - Choose visibility (public/private)
 - Initialize with README (optional)
2. Push your Python application code (structure something like: app.py, requirements.txt, possibly tests/ folder).

Visual aid: The flow starts at “Source” – when code is committed/pushed to GitHub.
(Refer image showing GitHub → pipeline trigger; e.g. #0image4 or #0image7)

Step 2: Create AWS CodeBuild buildspec

To define how CodeBuild builds/tests your app, create a file buildspec.yml in repo root.
Example:

```
yaml

version: 0.2

phases:
  install:
    commands:
      - echo "Installing dependencies"
      - pip install -r requirements.txt
  pre_build:
    commands:
      - echo "Running tests"
      - pytest # or appropriate test command
  build:
    commands:
      - echo "Packaging app"
      - mkdir build_output
      - cp -R . build_output/
artifacts:
  files:
    - build_output/**/*
```

- **install:** get dependencies
- **pre_build/test:** run unit tests
- **build:** package / prepare for deploy
- **artifacts:** what output gets passed to next stage

Step 3: IAM Permissions Setup (Required for Pipeline + Build + Deploy)

Make sure:

- There's an IAM role for CodePipeline that has permissions to access GitHub (via OAuth/token), CodeBuild, S3 (for storing artifacts), CodeDeploy / Elastic Beanstalk or whatever you deploy to.
- IAM role for CodeBuild that allows it to pull code, install dependencies, write logs, upload artifacts, etc.
- If deploying to EC2 (with CodeDeploy) or other AWS services, necessary permissions (e.g. EC2 instances might need a role, agent installed).

Step 4: Create CodeBuild Project in AWS

In console → CodeBuild → Create project:

- **Name:** e.g. my-python-build
 - **Source:** “GitHub” (or CodePipeline) → your repo & branch
 - **Environment:** choose runtime (e.g. Ubuntu, Amazon Linux), image that supports Python, compute size
 - **Buildspec:** either use the one in your repo (buildspec.yml) or inline commands in console
 - **Artifacts:** define where the build artifact will go (often an S3 bucket)
-

Step 5: Create AWS CodePipeline

This orchestrates the flow:

1. Go to AWS Console → CodePipeline → **Create pipeline**
2. Pipeline name: e.g. python-app-ci-pipeline
3. Service role: let AWS create or choose existing IAM role
4. **Source Stage**
 - Provider: GitHub → connect via OAuth / token
 - Choose repository and branch (e.g. main or master)
5. **Build Stage**
 - Provider: AWS CodeBuild
 - Select the build project you created
6. **Deploy Stage** (optional)
 - Could be Elastic Beanstalk, AWS Lambda, EC2 via CodeDeploy, ECS, etc.
 - Configure accordingly
7. Add optional **Manual Approval** stage if you want a check before deployment to production

Review settings → Create pipeline.

Step 6: Trigger & Run the Pipeline

- Make a change in GitHub (e.g. modify some code, add test, etc.) → commit & push
 - CodePipeline detects the change (via webhook)
 - Pipeline runs:
 1. Source fetch
 2. Build via CodeBuild (installation, tests, packaging)
 3. If build passes, artifact moves forward
 4. Deploy if deployment configured
 - Monitor logs via AWS Console (CodeBuild logs, pipeline executions)
-

Step 7: Verify & Monitor

- After deployment, verify the deployed app is working (URL, endpoints)
- Check logs, metrics (CloudWatch, etc.)
- If any failure in tests or build, fix code and re-push
- Optional: set up notifications (SNS, Slack, email) for pipeline failures/success